

Architektura i Implementacja Zdecentralizowanego Ekosystemu Subskrypcji NFT: Kompleksowy Raport Techniczny

Streszczenie Wykonawcze

Transformacja modeli subskrypcyjnych z paradygmatu Web2 do Web3 stanowi fundamentalną zmianę w sposobie monetyzacji relacji twórca-fan oraz zarządzania uprawnieniami cyfrowymi. Podczas gdy tradycyjne platformy opierają się na scentralizowanych bazach danych do śledzenia uprawnień, proponowany system wykorzystuje niezmienność i transparentność technologii blockchain do tokenizacji dostępu. Niniejszy raport szczegółowo opisuje wyczerpujący projekt w pełni zintegrowanego ekosystemu subskrypcji opartego na Non-Fungible Tokens (NFT). System został zaprojektowany w celu obsługi dwóch odrębnych person użytkowników: **Twórcy**, który wymaga zaawansowanych narzędzi do zarządzania planami, analityki przychodów i angażowania społeczności, oraz **Fana**, który oczekuje bezproblemowego procesu zakupu, przejrzystości posiadanych aktywów i wymiernej użyteczności.

Główny mechanizm opiera się na odnawialnym NFT działającym jako cyfrowa przepustka. W przeciwieństwie do statycznych cyfrowych przedmiotów kolekcjonerskich, te subskrypcyjne NFT posiadają dynamiczne stany – Aktywny, Wygasły lub Okres Karencji – zarządzane przez logikę on-chain i wizualizowane poprzez dynamiczne metadane. Architektura integruje standard EIP-5643 dla subskrypcji oraz ERC-5192 dla właściwości Soulbound (niezbywalności), gwarantując, że korzyści członkowskie pozostają przypisane do konkretnego subskrybenta. Aby zniwelować barierę użyteczności między Web2 a Web3, infrastruktura płatnicza wykorzystuje podejście hybrydowe. Integruje ona interfejsy API płatności cyklicznych Circle dla subskrypcji opartych na walutach fiat (karty kredytowe) oraz zdecentralizowane sieci automatyzacji (Gelato/Chainlink) dla natywnych płatności krypto. Ten dwutorowy system finansowy zapewnia dostępność dla użytkowników spoza świata krypto, jednocześnie zachowując etos decentralizacji dla zaawansowanych użytkowników.

Niniejszy raport służy jako plan techniczny, obejmujący specyfikacje inteligentnych kontraktów, architektury automatyzacji backendu, generowanie dynamicznych metadanych oraz projekt interfejsu użytkownika. Odnosi się on do zniuansowanych wyzwań optymalizacji opłat transakcyjnych (gas), synchronizacji stanów i kompatybilności międzyłańcuchowej, dostarczając skalowalne rozwiązanie dla studium przypadku „CodeMaster” i szerszego rynku.

1. Kontekst Strategiczny i Architektura Konceptyjna

Ekonomia subskrypcji stała się dominującym modelem monetyzacji dla twórców treści, usług oprogramowania i liderów społeczności. Jednak obecna infrastruktura Web2 opiera się w dużej mierze na scentralizowanych "bramkarzach" (ang. gatekeepers) – bazach danych należących do platform takich jak Patreon czy Substack. W tych systemach „subskrypcja” użytkownika jest

jedynie wierszem w prywatnej tabeli SQL. Jeśli platforma zostanie zamknięta, zbanuje użytkownika lub odetnie twórcę od usług bankowych, relacja zostaje natychmiast zerwana. Użytkownik nie posiada niczego; jedynie wynajmuje dostęp.

Web3 i tokeny NFT obiecały przejście w kierunku własności. Wczesne projekty NFT z powodzeniem stokenizowały cyfrowe przedmioty kolekcjonerskie, pozwalając użytkownikom na posiadanie sztuki lub przedmiotów w grach. Jednak pierwsza generacja NFT nie zdołała uchwycić czasowej natury subskrypcji. Token ERC-721 jest statyczny; raz wyemitowany (wymintowany), istnieje na zawsze, chyba że zostanie spalony. Nie rozumie on z natury pojęć „okresów ważności” czy „powtarzalnych przychodów”. W konsekwencji wczesne „członkowskie NFT” były dożywotnimi przepustkami – modelem, który jest ekonomicznie niezrównoważony dla twórców potrzebujących stałych przychodów do finansowania bieżącej produkcji treści. Niniejszy raport nakreśla architekturę **Systemu Subskrypcji NFT**, który rozwiązuje tę rozbieżność. Poprzez osadzenie logiki czasowej w inteligentnym kontrakcie i opakowanie jej w doświadczenie użytkownika naśladujące znane interfejsy Web2, tworzymy system, w którym „Subskrypcja” jest aktywem posiadanym przez użytkownika, transparentnie weryfikowalnym na blockchainie, a jednocześnie zdolnym do wygasania i odnawiania.

1.1 Podstawowa Propozycja Wartości

Analiza rynku i dostępnych technologii wskazuje na fundamentalne korzyści dla obu stron transakcji:

Dla Twórcy (CodeMaster):

- **Bezpośrednia Własność:** Lista subskrybentów znajduje się on-chain, jest przenośna i nie jest zamknięta w "ogrodzonym ogrodzie" (walled garden) jednej platformy.
- **Programowalny Przychód:** Inteligentne kontrakty mogą wymuszać podziały tantiem, automatyzować wypłaty dla partnerów i obsługiwać logikę dostępu warstwowego bez ręcznej interwencji.
- **Efektywność Kapitałowa:** Natychmiastowe rozliczanie płatności krypto (USDC) eliminuje 30-60 dniowe okresy oczekiwania na wypłatę, powszechne w platformach Web2.

Dla Fana (Jane):

- **Przejrzystość:** Zasady subskrypcji (cena, korzyści, limity podaży) są niezmiennym kodem, niepodlegającym arbitralnym zmianom regulaminu usług.
- **Tożsamość i Reputacja:** Subskrypcja typu „Soulbound” służy jako weryfikowalny dowód długoterminowego wsparcia, umożliwiając grywalizację nagród lojalnościowych (np. status „OG” dla użytkowników subskrybujących >1 rok).
- **Trwałość Aktywów:** Nawet po wygaśnięciu, NFT pozostaje w portfelu użytkownika jako pamiątka lub dowód przeszłego mecenatu, w przeciwieństwie do anulowanej subskrypcji Web2, która znika bez śladu.

1.2 Zakres Systemu i Definicja Produktu

System opisany w tym raporcie to platforma typu „Protokół jako Usługa”. Pozwala ona dowolnemu Twórcy (w naszym przypadku „CodeMaster”) wdrożyć własny, brandowany kontrakt subskrypcyjny bez pisania kodu. Obsługuje on pełny stos technologiczny:

1. **Inteligentne Kontrakty:** Logika on-chain zarządzająca stanem i czasem.
2. **Szyby Płatnicze:** Pomost między pieniędzmi (Fiat/Krypto) a czasem (Data Wygaśnięcia).
3. **Warstwa Wizualna:** Dynamiczna reprezentacja statusu subskrypcji (zmiana wyglądu NFT).

4. **Interfejsy:** Dashboardy do zarządzania i zakupu dla Twórcy i Fana. Poniższe sekcje zdekonstruują każdą z tych warstw, dostarczając technicznego planu wdrożenia.

2. Inżynieria Tokenów i Standardy Blockchain

Fundamentem każdego systemu subskrypcji NFT jest wybór odpowiednich standardów tokenów. Standardowy token ERC-721 jest niewystarczający dla subskrypcji cyklicznych, ponieważ brakuje mu wewnętrznego wymiaru czasu. W związku z tym architektura przyjmuje podejście kompozytowe, nakładając logikę czasową i restrykcje transferu na bazową specyfikację tokena niezamiennego.

2.1 EIP-5643: Standard Subskrypcji

Główny interfejs dla tego systemu jest zbudowany w oparciu o **EIP-5643**, standard zaprojektowany specjalnie dla odnawialnych subskrypcji NFT. Standard ten rozszerza ERC-721, wprowadzając znacznik czasu expiration bezpośrednio do pamięci kontraktu (storage).

Kluczowe zalety przyjęcia EIP-5643 zamiast autorskiej implementacji obejmują:

- **Interoperacyjność:** Portfele i marketplace'y obsługujące EIP-5643 mogą automatycznie wyświetlać status subskrypcji (np. „Ważne do 15.02.2025”) bez niestandardowej integracji.
- **Standaryzacja Zdarzeń:** Zdarzenie SubscriptionUpdate pozwala indeksom off-chain (takim jak The Graph) efektywnie śledzić odnowienia i anulowania.
- **Logika Odnawiania:** Standard wymusza funkcję renewSubscription, która przyjmuje tokenId i czas trwania. Tworzy to przewidywalny interfejs dla botów automatyzujących i bramek płatniczych w celu przedłużenia dostępu użytkownika.

W tym systemie funkcja renewSubscription jest zmodyfikowana tak, aby mogła być wywoływana nie tylko przez użytkownika, ale także przez autoryzowanych „operatorów” (procesory płatności lub kontrakty automatyzacji), pod warunkiem zweryfikowania płatności. To odwrócenie kontroli jest krytyczne dla zautomatyzowanych płatności cyklicznych.

2.2 ERC-5192: Implementacja Soulbound Token (SBT)

Krytycznym wymogiem biznesowym dla modeli subskrypcyjnych jest zapobieganie arbitrażowi korzyści. Jeśli NFT subskrypcyjny jest zbywalny, użytkownik mógłby kupić „Złoty Plan” po starej cenie i odsprzedać go na rynku wtórnym lub przenieść NFT do znajomego, gdy nie korzysta z benefitów (analogia do współdzielenia haseł). Aby temu zapobiec, system wdraża **ERC-5192: Minimal Soulbound NFTs**.

ERC-5192 wprowadza stan „zablokowany” (locked) do NFT. Dla tego systemu subskrypcji mechanizm blokowania działa następująco:

- **Przy Mintowaniu:** NFT jest emitowany ze statusem locked, co uniemożliwia operacje transferFrom i safeTransferFrom.
- **Wiązanie Korzyści:** Zapewnia to, że historia subskrypcji (np. „Członek od 2024 roku”) pozostaje przypisana do pierwotnej tożsamości, zwiększając wartość reputacyjną „Duszy” (portfela użytkownika).
- **Scenariusze Odzyskiwania:** Chociaż generalnie niezbywalny, administrator kontraktu (Twórca) zachowuje przywilej rescue lub unlock do obsługi przypadków kompromitacji

portfela, pozwalając na migrację subskrypcji na nowy adres w skrajnych scenariuszach obsługi klienta.

2.3 EIP-5006: Zarządzanie i Delegacja

Wymagania techniczne wspominają o możliwości implementacji **EIP-5006**. Jest to rozszerzenie ERC-1155, które wprowadza koncepcję „Użytkownika” (User) oddzielnego od „Właściciela” (Owner). Chociaż nasz system bazuje na ERC-721/EIP-5643, adaptacja logiki z EIP-5006 może być kluczowa w przyszłości dla subskrypcji korporacyjnych (gdzie firma jest właścicielem, a pracownik użytkownikiem benefitów). Jednakże dla uproszczenia modelu "Fan-Twórca", logika expiresAt z EIP-5643 jest wystarczająca i bardziej bezpośrednia. EIP-5006 może być jednak użyty jako warstwa zarządzania (Governance SBT) dla długoterminowych posiadaczy, dając im prawo głosu w DAO twórcy bez konieczności przekazywania tokena.

3. Architektura Inteligentnych Kontraktów

Warstwa on-chain jest źródłem prawdy dla praw dostępu. Architektura wykorzystuje model „Hub-and-Spoke”, gdzie centralna fabryka **SubscriptionFactory** tworzy indywidualne kontrakty **SubscriptionCollection** dla każdego Twórcy. Izoluje to stan i pozwala twórcom posiadać adresy swoich kontraktów.

3.1 Interfejsy Kontraktów i Struktury Danych

Główny kontrakt implementuje IERC5643, IERC5192 oraz IERC721Metadata.

Zmienne Stanu: Zamiast przechowywać dane w luźnych zmiennych, stosujemy mapowania dla efektywności:

- mapping(uint256 => uint64) private _expirations; // Mapuje TokenID na znacznik czasu Unix
- mapping(uint256 => uint256) public planId; // Mapuje TokenID na konkretne ID Planu (Srebrny, Złoty)
- mapping(uint256 => SubscriptionPlan) public plans; // Struktura definiująca szczegóły planu

Struktura: SubscriptionPlan Poniższa tabela przedstawia strukturę danych planu, zoptymalizowaną pod kątem zużycia gazu (gas packing):

Typ Danych	Nazwa Pola	Opis
string	name	Nazwa planu (np. "Złoty Fan")
uint256	price	Cena w stabilnej walucie (USDC, 6 miejsc po przecinku)
uint64	duration	Okres w sekundach (np. 2592000 dla 30 dni)
bool	active	Flaga dostępności (false = archiwizacja)
string	metadataURI	Bazowe URI dla wyglądu planu
uint32	maxSupply	Limit podaży (0 dla nielimitowanej)
uint32	currentSupply	Licznik aktualnych subskrybentów

3.2 Zarządzanie Czasem i Logika Wygaśnięcia

Solidity polega na `block.timestamp` do obliczeń czasu. Chociaż górnicy mogą nieznacznie manipulować tą wartością, granularność subskrypcji (dni/miesiące) sprawia, że 15-sekundowe odchylenia są nieistotne.

Funkcja `renewSubscription`: Ta funkcja jest silnikiem systemu. Musi ona:

1. Zweryfikować, czy wywołujący jest autoryzowany (Bramka Płatności lub Właściciel).
2. Sprawdzić, czy dostarczona płatność (`msg.value` lub transfer ERC-20) odpowiada aktualnej cenie planu.
3. Zaktualizować `_expirations[tokenId]`.

Logika Odnawiania:

- **Scenariusz A (Przedłużenie):** Jeśli subskrypcja jest obecnie aktywna (`_expirations[tokenId] > block.timestamp`), system dodaje czas trwania do *istniejącej* daty wygaśnięcia. Zapobiega to „traceniu” dni przez użytkowników odnawiających wcześniej.
- **Scenariusz B (Reaktywacja):** Jeśli subskrypcja wygasła, nowa data jest liczona od *bieżącego* czasu (`block.timestamp + duration`). Zapobiega to sytuacji, w której użytkownik płaciłby za czas przeszły, w którym nie miał dostępu.

3.3 Kontrola Dostępu i Role

Aby obsłużyć funkcje dashboardu opisane w wymaganiach, wymagana jest solidna lista kontroli dostępu (ACL). Wykorzystujemy AccessControl od OpenZeppelin.

- **DEFAULT_ADMIN_ROLE:** Twórca (CodeMaster). Może modyfikować szczegóły planu (opis, metadane), ale *nie może* zmieniać ceny dla istniejących aktywnych subskrypcji (zabezpieczenie przed rug-pull).
- **OPERATOR_ROLE:** Przypisana do botów automatyzacji (Gelato/Chainlink) oraz portfela przekąźnikowego (relayer) procesora płatności fiat. Ta rola pozwala wywoływać `renewSubscription` bez transferu środków on-chain, jeśli środki zostały pobrane off-chain (scenariusz fiat).
- **BENEFIT_MANAGER_ROLE:** Portfel API, który może odpytywać kontrakt w celu walidacji użytkowników dla dostępu do Discorda lub treści.

4. Infrastruktura Płatności Cyklicznych

Wymagania identyfikują integrację płatności jako kluczowe wyzwanie. System musi obsługiwać zarówno tradycyjne płatności kartą (dla masowej adopcji), jak i płatności krypto-natywne. Projektujemy **Dwutorową Bramkę Płatniczą**.

4.1 Tor Fiat: Integracja API Circle i Relayer

Dla użytkownika „Jane” płacącego 30 USD miesięcznie kartą, interakcja z blockchainem musi być wyabstrahowana. Wykorzystujemy **Circle's Programmable Wallets and Payments API**.

Przepływ Pracy (Workflow):

1. **Inicjacja:** Jane wprowadza dane karty w Panelu Fana.
2. **Przetwarzanie:** Circle procesuje obciążenie 30 USD.
3. **Webhook:** Circle wysyła `webhook payment_succeeded` do naszego Serwisu Backendowego.

4. **Akcja Relayera:** Backend waliduje podpis webhooka i inicjuje transakcję przez Portfel Relayer (posiadający OPERATOR_ROLE).
5. **Egzekucja On-Chain:** Relayer wywołuje adminRenew(tokenId, duration) na Inteligentnym Kontrakcie. Ta funkcja aktualizuje datę wygaśnięcia bez wymagania transferu tokenów on-chain od Jane.
6. **Cykliczność:** API subskrypcji Circle obsługuje comiesięczne obciążenia. Po każdym sukcesie kroki 3-5 są powtarzane.

Obsługa błędów: Jeśli karta zostanie odrzucona, webhook nie zostanie wysłany, data wygaśnięcia on-chain nie zostanie zaktualizowana, a NFT naturalnie przejdzie w stan „Wygasły” na blockchainie.

4.2 Tor Krypto: Zautomatyzowane Płatności "Pull"

Dla użytkowników płacących w krypto (np. USDC), nie możemy polegać na ręcznym podpisywaniu transakcji co miesiąc. Musimy użyć modelu **Zatwierdzenie (Allowance) + Automatyzacja**.

Infrastruktura: Gelato Network / Chainlink Automation.

1. **Zatwierdzenie (Jednorazowe):** Użytkownik zatwierdza Kontrakt Subskrypcji do wydawania USDC z jego portfela. Kluczowe jest użycie zatwierdzenia na wysoką kwotę (infinite approval) lub podpisów **ERC-2612 Permit** dla autoryzacji konkretnych cyklicznych potrąceń.
2. **Rejestracja Zadania:** Zadanie jest rejestrowane w Gelato/Chainlink w celu monitorowania mapowania _expirations.
3. **Warunek Sprawdzający:** Węzeł automatyzacji uruchamia funkcję widoku checkUpkeep():
 - Czy _expirations[tokenId] < block.timestamp + 3 dni?
 - Czy użytkownik ma wystarczające saldo USDC?
 - Czy allowance jest wystarczający?
4. **Egzekucja:** Jeśli checkUpkeep zwraca prawdę, węzeł automatyzacji wywołuje renewSubscription.
5. **Logika Płatności:** Kontrakt wykonuje USDC.transferFrom(user, creator, price). Jeśli to się nie uda (brak środków), transakcja zostanie cofnięta (revert), a subskrypcja ostatecznie wygaśnie.

Alternatywa: Superfluid Streaming Dla bardziej zaawansowanego doświadczenia krypto, system może zintegrować **Superfluid**. Zamiast dyskretnych miesięcznych płatności, użytkownik otwiera „Strumień” USDCx.

- *Mechanizm:* Kontrakt monitoruje strumień. Dopóki flowRate >= Cena Planu, funkcja isSubscriptionActive zwraca true.
- *Zalety:* Płatność co do sekundy; brak transakcji „odnawiania” (oszczędność gas).
- *Wady:* Wysoka złożoność dla przeciętnego użytkownika; wymaga owijania (wrapping) tokenów.
- *Rekomendacja:* Użycie dyskretnych płatności (Gelato) dla MVP, ponieważ odzwierciedla to znany model „miesięcznego rachunku” wymagany w opisie.

5. Dynamiczne NFT i Systemy Wizualne

Wymagania kładą nacisk na to, że wygląd NFT powinien zmieniać się w zależności od stanu

(Aktywny vs Wygasły) oraz czasu trwania (Poziom/Streak). Wymaga to rozwiązania typu **Dynamic Metadata**.

5.1 Architektura Metadanych

Statyczne linki IPFS są tu niewystarczające. Metadane muszą odzwierciedlać stan on-chain w czasie rzeczywistym. Proponujemy model **Hybrydowy On-Chain/Off-Chain**. Funkcja tokenURI w inteligentnym kontrakcie wskazuje na scentralizowany (ale weryfikowalny) punkt końcowy API: <https://api.platform.com/metadata/{tokenId}>.

Struktura Atrybutów w JSON:

```
{
  "name": "CodeMaster VIP - Token #42",
  "description": "Bilet dostępu do mentoringu VIP",
  "image": "https://api.platform.com/render/42.png",
  "animation_url": "https://api.platform.com/render/42.glb",
  "attributes":
}
```

5.2 Dynamiczne Generowanie Obrazu (Server-Side)

Gdy marketplace (OpenSea) lub portfel żąda obrazu, backend generuje go "w locie".

Stos Technologiczny: Node.js z bibliotekami **Sharp** lub **Canvas**.

Logika Generowania:

1. **Odpytanie Łańcucha:** API odpytuje blockchain o `_expirations[tokenId]` oraz `startBlock`.
2. **Determinacja Stanu:**
 - Jeśli teraz > wygaśnięcie: Renderuj szablon "Szary/Wygasły" ze stemplem "WYGASŁY".
 - Jeśli teraz < wygaśnięcie: Renderuj szablon "Złoty/Świecący".
 - *Obliczenie Streaku:* Oblicz (teraz - dataRozpoczęcia) / 30 dni. Wyrenderuj odpowiednią odznakę (np. Korona "3 Miesięczny Weteran").
3. **Kompozycja:** Używając Sharp, nałóż logo Twórcy, tło Planu, odznakę Streaku i tekst "Ważny do" na bazowe płótno.
4. **Cache'owanie:** Wygenerowany obraz jest buforowany na CDN (Cloudflare) z krótkim TTL (Time To Live), aby zapewnić wydajność przy zachowaniu świeżości danych.

5.3 Implementacja 3D (GLTF) i Tekstury

Zgodnie z wymaganiem, NFT może być "Kluczem" 3D. Wykorzystując modele **GLTF**, tworzymy trójwymiarowe bilety.

- **Implementacja:** Pole `animation_url` w metadanych wskazuje na plik `.glb`.
- **Dynamiczne Tekstutowanie:** Używamy bibliotek server-side do modyfikacji tekstury modelu 3D (np. nanosząc imię fana lub datę wygaśnięcia bezpośrednio na mapę tekstury 3D klucza) przed wysłaniem pliku GLTF do użytkownika. Pozwala to na wyświetlenie spersonalizowanego trójwymiarowego obiektu w portfelu lub metawersie.

6. Dashboard Twórcy: Doświadczenie i Architektura

Dashboard Twórcy to centrum dowodzenia. Musi ono abstrahować złożoność interakcji z inteligentnym kontraktem, zgodnie z sekcją A wymagań.

6.1 Kreator Planów (Interfejs Tworzenia)

Przepływ UI:

1. **Szczegóły Planu:** Twórca wypełnia formularz (Nazwa: "Złoty Fan", Cena: 30 USDC, Okres: Miesiąc).
2. **Studio Designu NFT:** Dashboard udostępnia edytor "Canva-like".
 - *Wybór Szablonu:* "Bilet", "Karta Członkowska", "Moneta 3D".
 - *Personalizacja:* Upload logo, wybór kolorów marki (branding CodeMaster).
 - *Podgląd Stanów:* Przełącznik "Widok Aktywny" / "Widok Wygasły", aby zobaczyć, jak NFT zachowa się po anulowaniu subskrypcji.
3. **Wdrożenie:** Kliknięcie "Opublikuj" wyzwala transakcję do SubscriptionFactory, która tworzy nową strukturę planu on-chain.

Szczegół Techniczny: Metadane planu (opis, grafika szablonu) są uploadowane na IPFS przy użyciu usługi takiej jak **Pinata** lub **NFT.Storage**. Hash CID jest zapisywany w mapowaniu plans kontraktu.

6.2 Analityka Subskrypcji (Warstwa Danych)

Twórcy potrzebują metryk w stylu Web2 (MRR, Churn). Dane blockchainowe są surowe; potrzebujemy **Indeksra**.

Infrastruktura:

- **The Graph:** Subgraph monitoruje zdarzenia Transfer, SubscriptionUpdate (Odnowienie) oraz SubscriptionCancel (choć to drugie jest rzadkie w modelu on-chain, częściej jest to po prostu brak odnowienia).
- **Obliczanie Metryk:**
 - *MRR (Miesięczny Powtarzalny Przychód):* Suma price dla wszystkich tokenów ze statusem Aktywny.
 - *Churn Rate (Wskaźnik Rezygnacji):* Odsetek tokenów, które przeszły ze stanu Aktywny do Wygasły w ciągu ostatnich 30 dni bez odnowienia.
- **Lista Subskrybentów:**
 - Tabela z kolumnami: Awatar Fana (ENS/Lens), Adres Portfela, Plan, Status, Data Następnej Płatności, Całkowita Wartość Wpłat.
 - *Źródło Danych:* Połączone dane z The Graph (status on-chain) i wewnętrznej bazy danych (nazwa użytkownika Discord, email jeśli podano).

6.3 System Integracji Benefitów

Sekcja "Integracje" pozwala łączyć korzyści off-chain z aktywami on-chain.

Integracja z Discordem:

1. **Mechanizm Bota:** Bot Discord ("SubBot") jest dodawany do serwera Twórcy.
2. **Weryfikacja:** Fan łączy portfel na Discordzie poprzez podpisanie wiadomości

(Collab.Land lub własne rozwiązanie).

3. **Przydzielanie Roli:** Bot odpytuje Kontrakt: `isSubscriptionActive(tokenId)`. Jeśli prawda, nadaje rolę "Złoty Fan".
4. **Obsługa Wygaśnięcia:** Bot uruchamia nocny cron job sprawdzający status. Jeśli subskrypcja wygasa, rola jest automatycznie usuwana.

Bramkowanie Treści (Webhook):

1. Twórca ustawia "Tajny URL Treści" (np. link do prywatnego wideo).
2. Platforma generuje link pośredniczący. Gdy fan w niego klika, platforma prosi o podpis portfela.
3. Backend waliduje podpis i status subskrypcji. Jeśli aktywny, przekierowuje do tajnej treści.

7. Interfejs Fana: Zakup i Zarządzanie

Doświadczenie Fana musi minimalizować "tarcie krypto" (crypto friction), zgodnie z sekcją B wymagań.

7.1 Proces Zakupu (Modal Płatności)

1. **Wybór:** Fan klika "Subskrybuj" na profilu CodeMastera.
2. **Wybór Planu:** Modal wyświetla porównanie "Srebrny" vs "Złoty" z listą korzyści.
3. **Metoda Płatności:**
 - *Zakładka 1: Karta (Fiat).* Formularz karty kredytowej. System estymuje opłaty gas (często subsydiowane przez platformę via meta-transakcje, aby obniżyć bariery wejścia).
 - *Zakładka 2: Krypto (USDC).* Wyświetla saldo portfela.
 - *Zatwierdzenie (Approve):* Pierwsza transakcja zatwierdzająca wydawanie USDC.
 - *Subskrypcja:* Druga transakcja mintująca NFT.
4. **Stan Sukcesu:** Animacja konfetti. NFT jest mintowany. Wyświetlany jest komunikat: "Witaj w klubie! Otrzymałeś NFT subskrypcji. Sprawdź swoje korzyści."

7.2 Panel "Moje Subskrypcje"

Znajdujący się w profilu Fana, panel ten naśladuje ekran zarządzania subskrypcjami Apple/Google.

Elementy UI:

- **Wizualizacja Kart:** Siatka kart NFT. Aktywne są nasycone kolorami i animowane; wygasłe są wyszarzone.
- **Widok Szczegółów:**
 - *Historia:* Tabela przeszłych płatności (hashe transakcji linkujące do Etherscan).
 - *Korzyści:* Bezpośrednie linki ("Dołącz do Discorda", "Odbierz Kod Rabatowy").
 - *Zarządzanie:*
 - **"Anuluj Subskrypcję":**
 - *Dla Krypto:* Anuluje zadanie w Gelato (przestaje pobierać środki).
 - *Dla Fiat:* Wywołuje API backendu, aby zatrzymać cykliczne obciążanie karty w Circle.
 - *Efekt Wizualny:* Komunikat "Czy na pewno? Stracisz dostęp do

korzyści po 15.02.2025". NFT pozostaje "Aktywny" do końca opłaconego okresu, potem zmienia stan na "Wygasły".

- **"Odnów Teraz"**: Ręczne przedłużenie wygasłej subskrypcji (reaktywacja NFT).

8. Infrastruktura Backendu i Automatyzacja

Podczas gdy blockchain przechowuje stan, backend koordynuje ekosystem (tzw. "Glue code").

8.1 Serwis "Watcher" (Cron Jobs)

Solidny serwis backendowy (Node.js/Go) jest wymagany do monitorowania zdrowia systemu i wyzwalania komunikacji.

- **Ostrzeżenie 3-Dniowe:**
 - *Zadanie:* Codzienne zapytanie do The Graph o subskrypcje wygasające w przedziale [teraz + 72h, teraz + 96h].
 - *Akcja:* Wysłanie e-maila/pusha: "Twój dostęp do CodeMaster wygasa za 3 dni. Upewnij się, że masz środki na koncie!".
- **Marker Wygaśnięcia:**
 - *Zadanie:* Sprawdzanie subskrypcji, które wygasły > 24h temu.
 - *Akcja:* Wyzwolenie webhooka do bota Discord, aby odebrać role. Aktualizacja statusu w wewnętrznej bazie danych na "Churned".

8.2 Ponawianie Płatności (Dunning)

Dla płatności krypto typu "pull" (Gelato), transakcje mogą się nie udać z powodu braku gasu lub środków.

- **Logika:** Jeśli transakcja odnowienia zostanie cofnięta (revert), system oznacza subskrypcję jako "Płatność Nieudana".
- **Windykacja (Dunning):** Backend wysyła alert do użytkownika. System automatyzacji ponawia próbę raz dziennie przez 3 dni (Okres Karencji). Jeśli nadal się nie udaje, zadanie automatyzacji jest anulowane, aby oszczędzać gas, a subskrypcja wygasa.

9. Studium Przypadku: Wdrożenie CodeMaster VIP

Scenariusz: Twórca "CodeMaster" uruchamia plan "VIP Mentor" (30 USD/msc) oferujący Code Review (korzyść manualna) i Prywatny Discord (korzyść automatyczna).

Wdrożenie:

1. CodeMaster wchodzi do Panelu Twórcy, łączy portfel.
2. Definiuje plan. Uploaduje model 3D "Złoty Klucz".
3. System wdraża kontrakt proxy SubscriptionCollection.
4. CodeMaster konfiguruje bota Discord w ustawieniach dashboardu.

Podróż Użytkownika (Jane):

1. **1 Stycznia:** Jane płaci 30 USD kartą. System mintuje Token #101. Wygaśnięcie: 1 Lutego. Wizualizacja: Złoty Klucz z poświatą "Aktywny".
2. **15 Stycznia:** Jane wchodzi na Discord. Bot weryfikuje, że Token #101 jest aktywny.
3. **31 Stycznia:** Circle obciąża kartę Jane na 30 USD. Sukces.

4. **1 Lutego:** Relayer aktualizuje wygaśnięcie Tokena #101 na 1 Marca.
5. **20 Lutego:** Jane anuluje subskrypcję w dashboardzie. Płatności Circle są zatrzymywane.
6. **1 Marca:** Token #101 wygasa. Wizualizacja zmienia się na "Zardzewiały Klucz". Pole status w UI pokazuje "Wygasty". Bot Discord usuwa rolę VIP Jane. NFT pozostaje w jej portfelu jako pamiątka (zgodnie z wymaganiem: "Jane może go zachować jako pamiątkę").
7. **10 Kwietnia:** Jane tęskni za społecznością. Klika "Odnów" na Zardzewiałym Kluczu. Tym razem płaci USDC. Token #101 jest reaktywowany (wygaśnięcie ustawione na 10 Maja), a wizualizacja wraca do Złotego Stanu.

10. Wyzwania Techniczne i Mapa Drogowa

10.1 Bezpieczeństwo Kontraktów

- **Re-entrancy:** Funkcja `renewSubscription` wchodzi w interakcje z zewnętrznymi tokenami (USDC). Stosujemy wzorzec Checks-Effects-Interactions oraz modyfikator `ReentrancyGuard`.
- **Manipulacja Czasem:** Poleganie na `block.timestamp`. Zabezpieczenie: 15-sekundowa zmienność jest akceptowalna dla subskrypcji miesięcznych.

10.2 Ryzyka Platformy

- **Falszowanie Metadanych:** Jeśli scentralizowane API metadanych padnie, NFT stracą wizualizację. Zabezpieczenie: Kontrakt pozwala na `updateBaseURI` przez administratora, ale wprowadza to zaufanie do centralizacji. Idealnie, krytyczne dane (data wygaśnięcia) są czytane bezpośrednio z kontraktu przez frontend, używając metadanych tylko do wizualizacji.
- **Zarządzanie Kluczami:** Portfel Relayer (dla płatności fiat) posiada rolę OPERATOR. W przypadku kompromitacji mógłby odnawiać subskrypcje za darmo. Zabezpieczenie: Ta rola nie powinna mieć możliwości zmiany cen ani wypłaty środków, jedynie przedłużania dat. Użycie Multi-sig dla `DEFAULT_ADMIN`.

10.3 Podsumowanie Technologiczne

Komponent	Technologia / Standard	Cel
Standard Aktywów	EIP-5643 + ERC-5192	Odnawialna, niezbywalna zdolność subskrypcyjna.
Smart Kontrakty	Solidity 0.8.x	Logika mintowania, wygasania i kontroli dostępu.
Płatności Fiat	Circle API	Przetwarzanie kart kredytowych i wyzwalamie odnowień on-chain.
Płatności Krypto	Gelato / Chainlink	Automatyzacja cyklicznych płatności "pull" ERC-20.
Metadane	Node.js + Sharp/Canvas	Server-side generowanie dynamicznych wizualizacji.
Indeksowanie	The Graph	Zasilanie analityki dashboardu i

Komponent	Technologia / Standard	Cel
		historii użytkownika.
Przechowywanie	IPFS (Pinata)	Hostowanie statycznych aktywów (szablony planów, modele 3D).

Niniejszy raport potwierdza, że w pełni zintegrowany, przyjazny dla użytkownika system subskrypcji NFT jest nie tylko wykonalny, ale i osiągalny przy użyciu obecnych standardów infrastruktury, realizując wszystkie założenia projektowe z zapytania.

Cytowane prace

1. Reference implementation of ERC5192 Minimal Soulbound Tokens - GitHub, <https://github.com/attestate/ERC5192>
2. EIP-5643 Subscription NFTs - Ethereum Magicians, <https://ethereum-magicians.org/t/eip-5643-subscription-nfts/10802>
3. Automated Transactions - Gelato Docs, <https://docs.gelato.cloud/web3-functions/introduction/automated-transactions>
4. Rebuilding payments for Web 3: from crypto rules to regulated rails in Asia-Pacific (Part 3), <https://thepaypers.com/crypto-web3-and-cbdc/expert-views/rebuilding-payments-for-web-3-from-crypto-rules-to-regulated-rails-in-asia-pacific-part-3>
5. Reliable, high-performance smart contract automation - Chainlink, <https://chain.link/automation>
6. cygaar/ERC5643: Subscription NFT Smart Contracts - GitHub, <https://github.com/cygaar/ERC5643>
7. Soulbound NFTs: Should they be a separate standard instead of ERC-721 extension?, https://www.reddit.com/r/ethdev/comments/1q8mp6u/soulbound_nfts_should_they_be_a_separate_standard/
8. What is Soulbound Token (SBT)? Guide to Web3 Identity | Cube Exchange, <https://www.cube.exchange/what-is/soulbound-token>
9. EIP-5643 Subscription NFTs - Page 2 - EIPs - Fellowship of Ethereum Magicians, <https://ethereum-magicians.org/t/eip-5643-subscription-nfts/10802?page=2>
10. EIP-5643 Subscription NFTs - Page 3 - EIPs - Fellowship of Ethereum Magicians, <https://ethereum-magicians.org/t/eip-5643-subscription-nfts/10802?page=3>
11. Web3 APIs & SDKs | Circle, <https://www.circle.com/developer>
12. A Deep Dive Into Circle's Web3 Services Platform, <https://www.circle.com/blog/an-overview-of-circles-web3-services-platform>
13. Expanding our Developer Tools in Circle Account, <https://www.circle.com/blog/expanding-our-developer-tools-in-circle-account>
14. Automating Smart Contract Tasks: Using Keepers and Relayers to Schedule Transactions, <https://hackernoon.com/automating-smart-contract-tasks-using-keepers-and-relayers-to-schedule-transactions>
15. Superfluid Dashboard Overview - HELP CENTER, <https://help.superfluid.finance/en/articles/6362394-superfluid-dashboard-overview>
16. Superfluid Dashboard V2 is Live: A Radical Improvement in Money Streaming UX - Medium, <https://medium.com/superfluid-blog/superfluid-dashboard-v2-is-live-a-radical-improvement-in-money-streaming-ux-d418dcf75ee7>
17. sharp vs canvas vs jimp vs imagescript | Image Processing Libraries - NPM Compare, <https://npm-compare.com/canvas,imagescript,jimp,sharp>
18. Crafting Images in the Cloud: Using Sharp and Node-Canvas Inside a Docker Container | by George Lopez | Medium, <https://medium.com/@george.benjamin.lopez/crafting-images-in-the-cloud-using-sharp-and-node-canvas-inside-a-docker-container-ffad867720fa>
19. Sharp.js: The Best Node.js Image Framework Ever - Leapcell, <https://leapcell.io/blog/sharpjs-best-nodejs-image-framework>
20. How to dynamically overlay a texture from a GLTF model - Three.js - Stack Overflow, <https://stackoverflow.com/questions/52236033/how-to-dynamically-overlay-a-texture-from-a-gltf->

model-three-js 21. glTF Transform, <https://gltf-transform.dev/> 22. Create a Dynamic NFT, <https://docs.nftport.xyz/docs/how-to-create-dynamic-nfts> 23. Create an NFT and upload your metadata to IPFS - LogRocket Blog, <https://blog.logrocket.com/create-nft-upload-metadata-ipfs/>